

Architectural Pattern Selection

Justification

Problem Statement #04 — Student Project Portfolio & Showcase App

We chose Microservices Architecture for the Student Project Portfolio & Showcase System.

Reason 1: Independent Scaling for Bursty, Uneven Load

Public voting and leaderboard refreshes spike dramatically during the live expo, while profile creation and judge scoring stay comparatively steady. Microservices let us scale only the Voting Service and Leaderboard Service horizontally during peak hours, instead of over-provisioning an entire monolith or layer stack to absorb a load spike confined to one function.

Reason 2: Fault Isolation Across Independent Stakeholder Workflows

Student teams, judges, and the public use largely independent workflows. With microservices, a failure or deployment of the Voting Service does not take down profile editing or judging — each service fails and recovers independently. A Layered or Client-Server design would route everything through shared layers or a single server, so an issue in one feature risks degrading the whole system during the time-critical expo window.

Security Advantage

Isolating the Voting Service and Identity Verification Service into their own units creates a tightly scoped security boundary around the system's most sensitive data — voter identities and OTP verification. Only these services need that access, so we can apply stricter authentication and rate-limiting there to enforce strict anti-sybil protection, without extending that overhead to lower-risk services like Profile or Judging.

Performance Benefit

Because each service is independently deployable, we can replicate the Leaderboard Service and Voting Service during peak expo hours to keep API latency under the NFR-001 target of 300 ms, while leaving Profile and Judging at baseline scale. A Layered architecture would force the whole Business layer to be scaled as one unit, over-provisioning components that don't need the extra capacity.